Programação Estruturada em C

Aula 6 — Vetores, funções com vetores e strings

Prof. Me. Eng. Matheus Kotaki

A mudança desta aula

Até agora...

- variáveis guardam valores isolados
- for repete operações
- funções organizam responsabilidades

A partir de hoje

- uma estrutura guarda vários valores
- o índice identifica cada posição
- laços processam a coleção inteira

Vetor = vários valores do mesmo tipo organizados por posições.

Um problema antes da sintaxe

Precisamos registrar as temperaturas de 7 dias e depois calcular a média.

```
float t1, t2, t3, t4, t5, t6, t7;
```

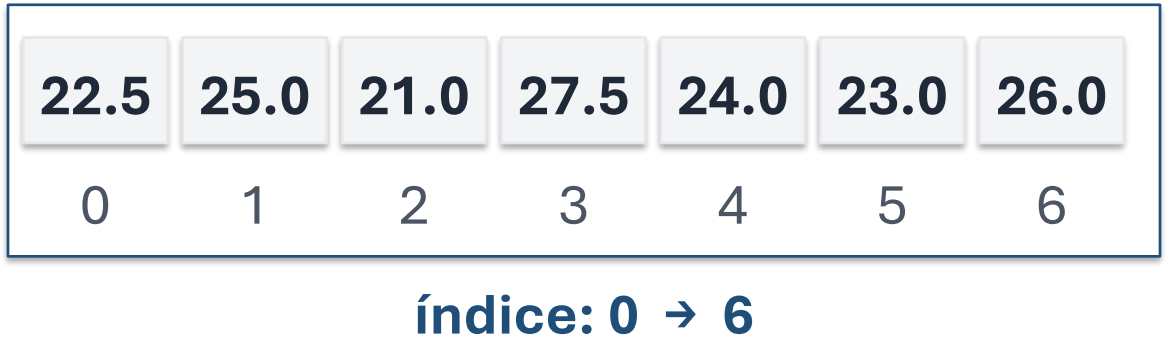
Funciona, mas como usar um for para processar t1, depois t2, depois t3...?

Precisamos de uma única estrutura cujas posições possam ser acessadas por um índice.

Vetor: uma única estrutura, várias posições

```
float temperaturas[7];
```

- 7 elementos
- todos do tipo float
- cada elemento possui um índice



Tamanho 7 não significa índice 7. O último índice é 6.

Vetores em C não são listas de Python

Semelhança útil

- agrupam vários valores
- permitem acesso por posição
- combinam naturalmente com laços

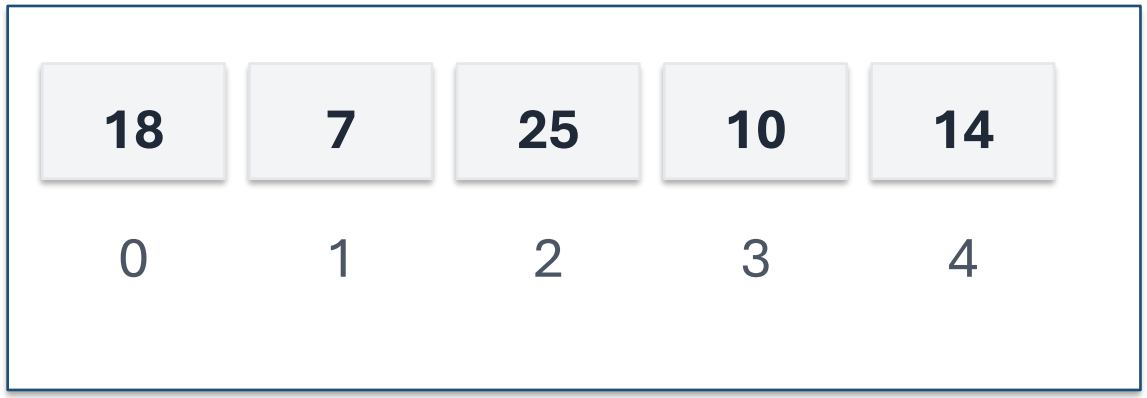
Diferenças importantes

- elementos têm o mesmo tipo
- tamanho é previamente definido
- não existe append()
- C não verifica automaticamente os limites

Índices: posição e valor

```
int valores[5] = {18, 7, 25, 10, 14};
```

```
valores[0] → 18
valores[2] → 25
valores[4] → 14
```



O índice começa em 0. Para TAM elementos: índices válidos de 0 a TAM - 1.

Acessar também permite alterar

```
int valores[4] = {10, 20, 30, 40};

valores[2] = 99;
printf("%d", valores[2]);
```

Antes

```
[ 10 ][ 20 ][ 30 ][ 40 ]
```

Depois

```
[ 10 ][ 20 ][ 99 ][ 40 ]
```

v[i] pode ser lido em uma expressão ou receber um novo valor.

Percorrer um vetor: o for ganha uma nova função

```
#include<stdio.h>
#define TAM 5

int main(){
   int v[TAM];

   for (int i = 0; i < TAM; i++)
   {
       scanf("%d", &v[i]);
   }
}
```

A cada repetição:

- `i` indica uma posição
- `v[i]` acessa o elemento dessa posição
- o mesmo código processa o vetor inteiro

O laço não “anda pelo vetor” sozinho: o índice é que determina qual posição será usada.

Erro clássico: <= TAM

```
for (int i = 0; i <= TAM; i++) {
    printf("%d ", v[i]);
}
```

Se TAM = 5, os valores de i são:

0 1 2 3 4 5

Mas o vetor possui apenas os índices 0, 1, 2, 3 e 4.

A condição usual é i < TAM.

C não protege automaticamente os limites

```
int v[5];
v[5] = 100;
```

O índice 5 não pertence ao vetor.

- o compilador pode não impedir
- o programa pode parecer funcionar
- o comportamento pode ser imprevisível

A responsabilidade de manter o índice dentro dos limites é do programa.

#define: dar nome ao limite

```
#define N 10

int valores[N];

for (int i = 0; i < N; i++) {
    ...
}
```

Por que usar N?

- evita repetir “números mágicos”
- mantém declaração e laços consistentes
- facilita alterar o limite depois

Nesta aula, #define é uma ferramenta para constantes simbólicas simples.

Processar não é apenas preencher e imprimir

Depois de armazenar os valores, podemos fazer perguntas sobre a coleção:

- somar / média
- maior / menor
- contar
- buscar

O padrão se repete: percorrer o vetor e decidir o que fazer com `v[i]`.

Rastreamento: índice, elemento e acumulador

```
int v[4] = {4, 7, 2, 8};
int soma = 0;

for (int i = 0; i < 4; i++){
    soma += v[i];
}
```

i	v[i]	soma
0	4	4
1	7	11
2	2	13
3	8	21

Rastrear ajuda a enxergar o vínculo entre `i`, `v[i]` e o estado do algoritmo.

Funções continuam sendo parte da solução

```
#include<stdio.h>
#define TAM 5

void preencher(int v[], int tam);
void exibir(int v[], int tam);
int somar(int v[], int tam);

int main() {
    int valores[TAM];
    preencher(valores, TAM);
    exibir(valores, TAM);
}
```

main() coordena

- cada função recebe uma responsabilidade
- o vetor pode ser usado por outras funções
- o tamanho é informado explicitamente

Por que passar também o tamanho?

```
void exibir(int v[], int tam) {
    for (int i = 0; i < tam; i++)
...
}
```

A função não “sabe” automaticamente quantos elementos deve percorrer.

Por isso, vetor e quantidade de elementos costumam viajar juntos.

O vetor usado pela função é o vetor original

```
void zerarPrimeiro(int v[]) {
    v[0] = 0;
}
```

No main:

```
int valores[3] = {8, 5, 2};
zerarPrimeiro(valores);
```

Antes

```
[ 8 ][ 5 ][ 2 ]
```

Depois

```
[ 0 ][ 5 ][ 2 ]
```

A função consegue acessar e alterar os elementos do vetor recebido.

Prática 1 — vetor + funções

Escreva um programa que leia 6 tempos (float), em segundos, obtidos em testes de uma tarefa. Depois, informe a média dos tempos e quantos resultados ficaram abaixo da média.

Organize o programa usando funções para:

- preencher o vetor
- calcular a média
- contar valores abaixo da média

Use #define para representar a quantidade de testes.

De vários números para vários caracteres

Um caractere:

```
char letra = 'A';
```

Vários caracteres:

```
char nome[30];
```

Uma string em C é representada em um vetor de char.

Strings entram aqui como uma aplicação importante de vetores e não como um novo tipo especial da linguagem.

Caractere, vetor de char e string

Caractere

```
char c = 'A';
```

Um único caractere.

Vetor de char

```
char texto[10];
```

Espaço para vários caracteres. Não é automaticamente um texto válido.

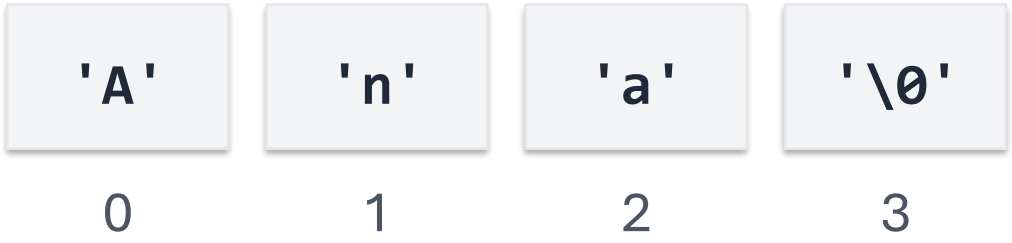
String

```
char nome[4] =
"Ana";
```

Sequência de caracteres encerrada por '\0'.

O terminador '\0'

A string "Ana" ocupa:



'\0' marca onde o texto termina.

Para guardar até 30 caracteres de texto, é preciso reservar espaço também para o terminador. Logo, precisa de 31 posições.

Por que scanf("%s", ...) não resolve tudo?

```
char titulo[50];
scanf("%s", titulo);
printf("%s", titulo);
```

Entrada digitada:

```
Introducao a Programacao
```

O %s para no primeiro whitespace.

```
Resultado: Introducao
```

fgets(): ler uma linha de texto

```
#include <stdio.h>

char titulo[50];
fgets(titulo, 50, stdin);
printf("%s", titulo);
```

O que informamos?

- onde armazenar: titulo
- limite de leitura: 50
- origem: stdin

fgets() pode ler espaços e respeita o limite informado.

Atenção: scanf() seguido de fgets()

```
int idade;
char nome[50];

scanf("%d", &idade);
fgets(nome, 50, stdin);
```

Se o usuário digita:

20↵

scanf() converte 20, mas a quebra de linha pode continuar disponível para a próxima leitura.

fgets() encontra o '\n' pendente e pode terminar imediatamente.

Consumindo o restante da linha quando necessário

```
int c;

int idade;
char nome[50];

scanf("%d", &idade);

while ((c = getchar()) != '\n' && c != EOF);
fgets(nome, 50, stdin);
```

O que esse laço faz?

Consome os caracteres restantes até encontrar a quebra de linha ou o fim da entrada.

Não é uma “limpeza mágica”: estamos consumindo conscientemente o restante da linha.

fgets() também pode armazenar a quebra de linha

Entrada:

Ana↵

O vetor pode conter:

```
'A'  'n'  'a'  '\n'  '\0'
```

Isso importa quando vamos comparar o texto.

"Ana\n" e "Ana" não possuem exatamente o mesmo conteúdo.

Nesta aula, basta reconhecer o problema; a manipulação de strings será retomada.

Podemos localizar a quebra de linha usando os índices do próprio vetor.

```
#define TAM 50

int main(){
   char nome[TAM];

   fgets(nome, TAM, stdin);

   for (int i = 0; nome[i] != '\0'; i++) {
       if (nome[i] == '\n') {
           nome[i] = '\0';
           break;
       }
   }
}
```

Antes

```
'A'  'n'  'a'  '\n'  '\0'
```

Depois

```
'A'  'n'  'a'  '\0'
```

O laço percorre os caracteres, encontra '\n' e o substitui por '\0'.

Uma forma mais prática: strcspn()

A biblioteca <string.h> oferece uma forma compacta de localizar o '\n'.

```
#include <string.h>
#define TAM 50

int main(){
   char nome[TAM];

   fgets(nome, TAM, stdin);
   nome[strcspn(nome, "\n")] = '\0';
}
```

1. **`strcspn(nome, "\n")`**
   localiza a posição do primeiro '\n'

2. **nome[posição]**
   acessa essa posição do vetor

3. **= '\0'**
   faz a string terminar antes da quebra de linha

strcspn() está disponível em <string.h>

Strings não são comparadas com ==

```
char a[20] = "azul";
char b[20] = "azul";

if (a == b) {
    ...
}
```

== não compara o conteúdo textual das duas strings.

Para comparar conteúdo:

```
strcmp(a, b)
```

strcmp() está disponível em <string.h>.

strcmp():

```
#include <string.h>

int main(){
   if (strcmp(a, b) == 0) {
       printf("Textos iguais");
   }
}
```

Nesta aula:

retorno 0 → strings iguais

Valores positivos e negativos existem, mas não são o foco agora.

O importante é abandonar a ideia de usar == para comparar o conteúdo.

Prática 2

Em uma competição acadêmica, uma apresentação recebe notas (float) de 8 avaliadores, de 0 a 10. Além das notas, o sistema deve registrar o título da apresentação.

O programa deverá:

- solicitar o título da apresentação, permitindo espaços;
- armazenar as 8 notas em um vetor;
- calcular a média das notas;
- identificar a maior e a menor nota;
- solicitar uma nota e informar quantas vezes ela aparece.

Prática 2 - Continuação

Ao final, apresente:

- o título da apresentação;
- todas as notas armazenadas;
- a média das notas;
- a maior e a menor nota;
- quantas vezes apareceu a nota pesquisada.

Requisitos de implementação

- Use #define para os tamanhos necessários.
- Use fgets() para ler o título.
- Retire o '\n' armazenado pelo fgets().
- Organize a solução em funções.
- Ao passar um vetor para uma função, informe também seu tamanho.
- Evite concentrar toda a lógica em main().

Antes de escrever o código, decida quais tarefas do programa devem ser transformadas em funções.

Prática 2 – Sugestão de Funções

```
void lerNotas(float notas[], int tam);
void exibirNotas(float notas[], int tam);
float calcularMedia(float notas[], int tam);
float encontrarMaior(float notas[], int tam);
float encontrarMenor(float notas[], int tam);
int contarAcimaMedia(float notas[], int tam, float media);
```